## Lab Introduction: Building and Deploying a Real DevOps Service

In this lab, you'll build and deploy a fully working mini-service that mirrors the kind of internal tooling real engineering teams rely on every day. By following the steps, you'll have created, containerised, and deployed a genuine service using a modern DevOps toolchain.

### What You'll Build

You'll create a Status Service, a small Node.js REST API that exposes a single endpoint:

**/status** – returns the service name, version, environment, and health information.

Although simple, this type of service is extremely common in real organisations. Teams use it for:

- Health checks
- Monitoring
- Environment validation
- As a template for new internal services
- How You'll Deploy It
- Your service will be:
- Containerised with Docker
- Built and deployed automatically using Jenkins

This mirrors the workflow used in real DevOps pipelines, giving you hands-on experience with the same tools and processes used in industry.

## Jenkins Pipelines Workshop

## Deploying a Real World Web App

In this workshop you will build, containerise, and deploy a small but realistic web application using Jenkins. This mirrors how many internal services are built and deployed in real organisations.

### The application

The app is a simple Node.js REST service called Status Service.

What it does:
- Exposes an HTTP endpoint /status
- Returns service name, version, and environment
- Designed to simulate an internal health or status service

This pattern is extremely common in real systems.

## Prerequisites

- Git
- Docker Desktop
- Java 17 or later
- Node.js 18 or later

## Step 1 Create the application

1. Create a folder called **status-service**
2. Inside it create a file called **app.js**
   Paste the following code

```
const express = require('express');
const app = express();

const PORT = 3000;
const SERVICE_NAME = 'status-service';
const VERSION = process.env.APP_VERSION || '1.0.0';
const ENV = process.env.ENV || 'dev';

app.get('/status', (req, res) => {
    res.json({
        service: SERVICE_NAME,
        version: VERSION,
        environment: ENV,
        status: 'OK'
    });
});

app.listen(PORT, () => {
    console.log(`Service running on port ${PORT}`);
});
```

## Step 2 Add package.json

Create a file called package.json and add:

```
{
    "name": "status-service",
        "version": "1.0.0",
            "main": "app.js",
                "scripts": {
        "start": "node app.js"
    },
    "dependencies": {
        "express": "^4.18.2"
    }
}
```

## Step 3 Test the app locally

Run:

**npm install**

**npm start**

Open a browser and go to **http://localhost:3000/status**
You should see a JSON response.

## Step 4 Create a Dockerfi321le

Create a file called Dockerfile and add:

```
FROM node: 18 – alpine
WORKDIR / app
COPY package.json.
RUN npm install--only = production
COPY app.js.
    EXPOSE 3000
CMD["npm", "start"]
```

## Step 5 Build and run container locally

**docker build -t status-service:1.0.0 .**

**docker run -p 3000:3000 status-service:1.0.0**

Verify the /status endpoint again.

## Step 6 Create Jenkinsfile

Create a **Jenkinsfile** in the root of the project:
Copy the text below

```
pipeline {
  agent any

  parameters {
        choice(name: 'ENV', choices: ['dev', 'test', 'prod'], description:
'Target environment')
    }

  stages {
        stage('Checkout') {
      steps { checkout scm }
        }

        stage('Build Image') {
      steps {
        sh 'docker build -t status-service:${BUILD_NUMBER} .'
            }
        }
```

```
        stage('Test') {
      steps {
        sh 'docker run --rm status-service:${BUILD_NUMBER} node -e
"console.log(\"Tests passed\")"'
            }
        }

        stage('Deploy') {
      when {
        expression { params.ENV != 'prod' }
            }
      steps {
        sh 'docker run -d -p 3000:3000 -e ENV=${ENV} status-
service:${BUILD_NUMBER}'
            }
        }

        stage('Deploy to Prod') {
      when {
        expression { params.ENV == 'prod' }
            }
      steps {
        input message: 'Approve production deployment'
        sh 'docker run -d -p 3000:3000 -e ENV=prod status-
service:${BUILD_NUMBER}'
            }
        }
    }
}
```

## Step 7 Run the pipeline

- Commit all files to Git.
- Create a Jenkins Pipeline job pointing at the repository.
- Run the pipeline and select an environment.
- Verify the service is running via /status.

🏁** End ** 🏁

## Well done! 👏🏻

Now you can confidently say:

*"I built a service, containerised it, and Jenkins deployed it."*

## What you've achieved

- You've built a real service
- Containerised it

- Automated build and deployment
- Used parameters and approvals

This is a genuine end to end DevOps workflow.

**You have also experienced:**

- how a REST API exposes operational information
- how environment changes reflected live through the /status endpoint

I hope this lab has given you a genuine DevOps outcome, not a simulation, but a real build-and-deploy workflow you control end-to-end. 👍🏻